Modern Policy Optimization (Trust Regions, TRPO, PPO)

Bias–Variance Tradeoff in Actor–Critic

- **Sources of Variance**
  - Monte Carlo returns G_t
  - Long trajectories
  - Stochastic rewards
  - Stochastic policies
  - Sparse or delayed rewards
- High variance makes REINFORCE unstable.

- **Sources of Bias**
  - Bootstrapped value estimates $V_\phi(s)$
  - Function approximation errors in the critic
  - Imperfect advantage estimates
  - Finite-sample training of the critic
- Small bias is often acceptable if variance is reduced substantially.

Method	Bias	Variance
Monte Carlo	none	very high
TD(0)	some	low
TD(λ), GAE(λ)	adjustable	adjustable

Actor–Critic dynamically balances **bias (from critic)** vs **variance (from MC)**.

Actor–Critic: Pseudocode

- On-Policy Actor–Critic Algorithm

Initialise

Policy parameters θ · Value function parameters ϕ

repeat

Generate trajectory $(s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

for $t = 0$ to $T-1$

Critic update (value function)

$$\delta_t = r_t + \gamma \cdot V_\phi(s_{t+1}) - V_\phi(s_t)$$

$$\phi \leftarrow \phi + \beta \cdot \delta_t \cdot \nabla_\phi V_\phi(s_t)$$

TD error δ_t shared

Actor update (policy)

$$\theta \leftarrow \theta + \alpha \cdot \delta_t \cdot \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

until convergence

Natural Gradient Motivation 1/2

Problem with Vanilla Policy Gradient

Standard update:

$$\theta \leftarrow \theta + \alpha g, \qquad g = \mathbb{E}[A^{\pi}(s,a)\,\nabla_{\theta}\log \pi_{\theta}(a\,|\,s)]$$

This uses **Euclidean geometry** in parameter space.

But:

- A small step in θ may produce a large change in the policy
- A large step in θ may produce a small change in the policy
- Learning becomes unstable and sensitive to parameterization

Natural Gradient Motivation 2/2

Natural Policy Gradient Idea

Use the geometry of **policy space**, not parameter space.

Define the **Fisher Information Matrix:**

$$F = \mathbb{E}[\nabla_\theta \log_\theta \pi_\theta(a \mid s) \nabla_\theta \log_\theta \pi_\theta(a \mid s)^T]$$

Natural gradient direction

$$\boxed{\tilde{g} = F^{-1} g}$$

This accounts for how changes in θ affect the distribution over actions

📘 Some Intuition

- Natural gradient moves the policy the same amount in distribution space regardless of parameterization
- Leads to more stable and efficient learning
- Forms the basis of TRPO and motivates the surrogate/ KL constraints of PPO

Advantage Estimation

Problems With Simple Advantage Estimators

Problems With Simple Advantage Estimators

1) **Monte Carlo Advantage**

$$\hat{A}_t = G_t$$

- Unbiased
- Extremely high variance, especially for long horizons
- Leads to unstable actor updates

2) **TD(0) Advantage**

$$\hat{A}_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- Low variance
- But biased, depends heavily on critic accuracy
- Short-term signal only (no multi-step credit)

Problems With Simple Advantage Estimators

Problems With Simple Advantage Estimators

1) **Monte Carlo Advantage**

$$\hat{A}_t = G_t$$

- Unbiased
- Extremely high variance, especially for long horizons
- Leads to unstable actor updates

2) **TD(0) Advantage**

$$\hat{A}_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- Low variance
- But biased, depends heavily on critic accuracy
- Short-term signal only (no multi-step credit)

Motivation for GAE (Generalized Advantage Estimation) [Schulman et al. (2015)]

GAE(λ) blends multiple n-step returns:

- Helps reduce variance
- Controls bias smoothly via parameter λ
- Provides a long-term but stable advantage estimator
- Dramatically improves PPO/A2C performance

Generalized Advantage Estimator

- TD Residuals (1-step Advantages): $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- GAE(λ) averages multi-step advantages with exponentially decaying weights:

$$\boxed{\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}}$$

📘 **Intuition**

This combines:
- TD(0) (when λ = 0)
- Monte Carlo advantages (when λ = 1)

Equivalent Truncated Form (finite horizon):

$$\hat{A}_t = \delta_t + \gamma\lambda \, \delta_{t+1} + \gamma^2\lambda^2 \, \delta_{t+2} + \cdots + \gamma^{T-t-1}\lambda^{T-t-1} \, \delta_{T-1} \, .$$

Generalized Advantage Estimator

- TD Residuals (1-step Advantages): $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- GAE(λ) averages multi-step advantages with exponentially decaying weights:

$$\boxed{\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}}$$

📘 **Intuition**

This combines:
- TD(0) (when λ = 0)
- Monte Carlo advantages (when λ = 1)

Equivalent Truncated Form (finite horizon):

$$\hat{A}_t = \delta_t + \gamma\lambda \, \delta_{t+1} + \gamma^2\lambda^2 \, \delta_{t+2} + \cdots + \gamma^{T-t-1}\lambda^{T-t-1} \, \delta_{T-1} \, .$$

Key Idea — GAE produces an advantage estimator that is:

- Low variance (due to λ < 1)
- Low bias (due to multi-step accumulation)
- Smooth (avoids TD jumps and MC noise)

GAE Bias–Variance Tradeoff

- GAE(λ) Blends Multi-Step Returns:

$$\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

λ controls how much future information is included in the advantage estimate.

Intuition:

- Small λ:
    - Critic dominates
    - Fast, stable learning
    - More bias due to reliance on V_ϕ
- Large λ:
    - More Monte Carlo structure
    - Lower bias, but increased variance
    - Sensitive to noise and reward sparsity

📘 **Why GAE Works Well:**

- **Smooths the variance spikes from MC returns**
- **Maintains long-horizon credit assignment**
- **Robust to critic imperfections**
- **Empirically optimal λ is typically 0.90–0.97 (Atari, MuJoCo, PPO, LLM PPO-KL/GRPO)**

Trust Regions, TRPO, PPO

Why Vanilla Policy Gradient Fails

- Update rule (single step): $\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
- Mini-batch estimate:

$$\theta \leftarrow \theta + \alpha \frac{1}{N} \sum_{t=1}^{N} \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

- Instability Issues: Large gradient steps can
  - Drastically change the policy
  - Collapse probability mass onto poor actions
- Mismatch: parameter space vs policy space
  - Small change in θ → large change in π_θ
  - Large change in θ → small change in π_θ
- No explicit control over:
  - KL divergence (old vs new policy)
  - Trust in gradient estimates

📘 Practical Symptomps:
- Highly sensitive to learning rate
- Training can diverge or oscillate
- Performance may sharply drop after a “bad” update
- Poor sample efficiency: Hard to reuse data across updates

Policy Improvement Lemma

- For any two policies π and π'

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}} \left[\mathbb{E}_{a \sim \pi'} \left[A^{\pi}(s, a) \right] \right]$$

- Interpretation
  - If actions chosen by π' have **positive advantage** under π → **performance improves**
  - If π' selects actions with **negative** A^{π} → **performance degrades**

Policy Improvement Lemma

- For any two policies π and π'

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}} \left[\mathbb{E}_{a \sim \pi'} \left[A^{\pi}(s, a) \right] \right]$$

- Interpretation
  - If actions chosen by π' have **positive advantage** under π → **performance improves**
  - If π' selects actions with **negative** A^{π} → **performance degrades**

Key Insight for TRPO: Goal $J(\pi') - J(\pi) > 0$

- **Lemma suggests:** Choose π' such that

$$\mathbb{E}_{s \sim d^{\pi'}} \left[\mathbb{E}_{a \sim \pi'} \left[A^{\pi}(s, a) \right] \right] > 0$$

Policy Improvement Lemma

- For any two policies π and π'

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}} \left[\mathbb{E}_{a \sim \pi'} \left[A^{\pi}(s, a) \right] \right]$$

- Interpretation
  - If actions chosen by π' have **positive advantage** under π → **performance improves**
  - If π' selects actions with **negative** A^{π} → **performance degrades**

Key Insight for TRPO: Goal $J(\pi') - J(\pi) > 0$

- **Lemma suggests:** Choose π' such that

$$\mathbb{E}_{s \sim d^{\pi'}} \left[\mathbb{E}_{a \sim \pi'} \left[A^{\pi}(s, a) \right] \right] > 0$$

- **Challenge:** $d^{\pi'}$ **is unknown (depends on the new policy)**

Implication: **TRPO** approximates this objective

Monotonic Improvement Bound 1/2

- From Kakade & Langford (2002), Schulman et al. (2015):

$$J(\pi') - J(\pi) \geq \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi}}\,\mathbb{E}_{a\sim\pi'}\left[A^{\pi}(s,a)\right] \;-\; \frac{2\gamma}{(1-\gamma)^2}\,\epsilon\, D_{\mathrm{TV}}(\pi\|\pi')$$

- **First term:** local surrogate improvement
- **Second term:** penalty for policy change

Where:

- D_{TV} is total variation distance
- $\epsilon = \max_{s,a} |A^{\pi}(s,a)|$
- The last term limits how far π' can deviate from π

Monotonic Improvement Bound 2/2

- As long as the policy change stays small, the bound guarantees: $J(\pi') \geq J(\pi)$

This is the theoretical justification for trust-region constraints.

Key Implications for TRPO

- We want to **maximize the surrogate:**

$$\mathbb{E}_{s \sim d^{\pi},\, a \sim \pi'}\left[A^{\pi}(s, a)\right]$$

- But we must **restrict policy updates:**

$$D_{\mathrm{TV}}(\pi \| \pi') \text{ must be small}$$

- KL divergence is used as a **practical, differentiable proxy**

Natural Gradient Interpretation (see the note)

TRPO Trust Region Constraint

$$\mathbb{E}_{s\sim d^{\pi_{old}}}\left[D_{\mathrm{KL}}\left(\pi_{\text{old}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s)\right)\right] \leq \delta$$

- For small parameter updates, linearizing KL gives:

$$D_{\mathrm{KL}} \approx \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}),$$ where F is the **Fisher Information Matrix** $F = \mathbb{E}\left[\nabla_\theta \log \pi(a \mid s)\, \nabla_\theta \log \pi(a \mid s)^\top\right]$

Natural Gradient Interpretation (see the note)

TRPO Trust Region Constraint

$$\mathbb{E}_{s\sim d^{\pi_{old}}}\left[D_{\mathrm{KL}}\left(\pi_{\text{old}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s)\right)\right] \leq \delta$$

- For small parameter updates, linearizing KL gives:

$$D_{\mathrm{KL}} \approx \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}),$$ where F is the **Fisher Information Matrix** $F = \mathbb{E}\left[\nabla_\theta \log \pi(a \mid s)\, \nabla_\theta \log \pi(a \mid s)^\top\right]$

Constrained Optimization → Natural Gradient Step

- TRPO solves:

$$\max_\theta \quad g^\top(\theta - \theta_{\text{old}}) \quad \text{s.t.} \quad \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}) \leq \delta$$

where: $g = \nabla_\theta L(\theta_{\text{old}})$ is the vanilla policy gradient.

Natural Gradient Interpretation (see the note)

TRPO Trust Region Constraint

$$\mathbb{E}_{s\sim d^{\pi_{old}}}\left[D_{\mathrm{KL}}\left(\pi_{\text{old}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s)\right)\right] \leq \delta$$

- For small parameter updates, linearizing KL gives:

$$D_{\mathrm{KL}} \approx \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}),$$ where F is the **Fisher Information Matrix** $F = \mathbb{E}\left[\nabla_\theta \log \pi(a \mid s)\, \nabla_\theta \log \pi(a \mid s)^\top\right]$

Constrained Optimization → Natural Gradient Step

- TRPO solves:

$$\max_\theta \quad g^\top(\theta - \theta_{\text{old}}) \quad \text{s.t.} \quad \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}) \leq \delta$$

where: $g = \nabla_\theta L(\theta_{\text{old}})$ is the vanilla policy gradient.

- This QP has closed-form solution:

$$\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g$$

which is exactly a Natural Gradient update scaled to satisfy the KL constraint.

Why TRPO Is Impractical

1. **Requires Second-Order Information**
   - Fisher matrix $F = \mathbb{E}[\nabla \log \pi \nabla \log \pi^\top]$
   - Huge, expensive, cannot store explicitly

2. **Constrained Optimization Step** $\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g$
   - Requires $F^{-1} g$ (natural gradient)
   - Solved via conjugate gradient (10–20 iterations)
   - Each step needs Fisher-vector products → high cost

3. **Needs Line Search**
   - Evaluate surrogate + KL constraint
   - Repeat multiple times → slow and brittle

4. **Hard to Scale** (breaks batching & GPU efficiency; requires CPU-bound CG loops)

Why TRPO Is Impractical

1. **Requires Second-Order Information**
   - Fisher matrix $F = \mathbb{E}[\nabla \log \pi \nabla \log \pi^\top]$
   - Huge, expensive, cannot store explicitly

2. **Constrained Optimization Step** $\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g$
   - Requires $F^{-1} g$ (natural gradient)
   - Solved via conjugate gradient (10–20 iterations)
   - Each step needs Fisher-vector products → high cost

3. **Needs Line Search**
   - Evaluate surrogate + KL constraint
   - Repeat multiple times → slow and brittle

4. **Hard to Scale** (breaks batching & GPU efficiency; requires CPU-bound CG loops)

Takeaway: ✔ Strong theory (monotonic improvement, trust region); ✖ Too expensive for modern deep RL / LLMs
- TRPO is elegant but slow → PPO replaces it

Proximal Policy Optimization (PPO)

Motivation for PPO: Making TRPO Practical

The Problem With TRPO

- TRPO gives strong theory:
  - monotonic improvement bound
  - natural gradient step
  - KL trust region constraint

…but it is computationally heavy:

  - requires conjugate gradient (CG)
  - requires Fisher-vector products
  - requires line search
  - hard to scale to deep networks or large batches
  - too slow / brittle for practical training (esp. LLMs)

Motivation for PPO: Making TRPO Practical

Goal of PPO (Schulman et al., 2017)

- Design a method that:
  - preserves TRPO's stability
  - avoids large destructive policy updates
  - controls deviation via a KL-like mechanism
  - **BUT uses only:**
    - first-order gradients
    - simple SGD
    - minibatches
    - vectorized operations
    - no CG, no Hessian approximations
    - no line search

Motivation for PPO: Making TRPO Practical

PPO Core Idea

- Replace TRPO's constrained optimization:

$$\max_{\theta} \; J(\theta) \quad \text{subject to} \quad \mathbb{E}\Big[D_{\mathrm{KL}}\big(\pi_{\text{old}}(\,\cdot \mid s) \,\|\, \pi_{\theta}(\,\cdot \mid s)\big)\Big] \leq \delta$$

with a penalized or clipped surrogate objective that:

- discourages large policy updates
- keeps the policy inside a “soft trust region”
- is cheap to compute via vanilla SGD

Importance Sampling Ratio

Why We Need Importance Sampling

- Data are collected using the old policy π_{old}
- We want to optimize a new policy π_θ
- Directly using on-policy gradients would require fresh rollouts after every update (expensive)
- Importance sampling corrects for this mismatch

Importance Sampling Ratio

Why We Need Importance Sampling

- Data are collected using the old policy π_{old}
- We want to optimize a new policy π_θ
- Directly using on-policy gradients would require fresh rollouts after every update (expensive)
- Importance sampling corrects for this mismatch

Importance Sampling Ratio:

$$\boxed{r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}}$$

This ratio tells us how much more (or less) likely the new policy assigns probability to the sampled action.

Importance Sampling Ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Key Properties

- $r_t = 1$: new policy matches old policy

- $r_t > 1$: action is more likely under π_θ

- $r_t < 1$: action is less likely under π_θ

For small policy updates, $r_t \approx 1$

Importance Sampling Ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Key Properties

- $r_t = 1$: new policy matches old policy

- $r_t > 1$: action is more likely under π_θ

- $r_t < 1$: action is less likely under π_θ

For small policy updates, $r_t \approx 1$

Rewriting the Policy Gradient

Using importance sampling:

$$J(\theta) - J(\theta_{old}) \approx \mathbb{E}_{s_t, a_t \sim \pi_{\text{old}}} \left[r_t(\theta) A_t \right]$$

This is the starting point for PPO's surrogate objective.

PPO Surrogate Objective

Surrogate Objective Using Importance Sampling

- Using data collected from π_{old}, define:

$$L^{PG}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{old}}[r_t(\theta) A_t]$$

Connection to Policy Gradient Theorem

This objective is a first-order approximation of the true policy gradient objective around π_{old}

$$J(\pi_\theta) \approx J(\pi_{\text{old}}) + \mathbb{E}[r_t(\theta) A_t]$$

PPO Surrogate Objective

Surrogate Objective Using Importance Sampling

- Using data collected from π_{old}, define:

$$L^{PG}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{old}}[r_t(\theta) A_t]$$

Connection to Policy Gradient Theorem

This objective is a first-order approximation of the true policy gradient objective around π_{old}

$$J(\pi_\theta) \approx J(\pi_{\text{old}}) + \mathbb{E}[r_t(\theta) A_t]$$

Key Limitation:

- Nothing prevents $r_t(\theta)$ from becoming **very large**

- Large ratios → extremely large policy updates → instability

Unconstrained Ratio ⇒ Catastrophic Policy Updates

Surrogate objective:

$$L^{\mathsf{PG}}(\theta) = \mathbb{E}\left[r_t(\theta)\, A_t\right], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathsf{old}}(a_t \mid s_t)}$$

Issue: unconstrained importance ratios

- No constraint on $r_t(\theta)$
- During SGD:
  - $r_t \gg 1$ for some actions
  - $r_t \ll 1$ for others

Unconstrained Ratio ⇒ Catastrophic Policy Updates

Surrogate objective:

$$L^{\text{PG}}(\theta) = \mathbb{E}\left[r_t(\theta)\, A_t\right], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Issue: unconstrained importance ratios

- No constraint on $r_t(\theta)$
- During SGD:
  - $r_t \gg 1$ for some actions
  - $r_t \ll 1$ for others

Failure modes

- Probability mass collapses onto few actions
- Loss of exploration
- Large KL: $\text{KL}(\pi_{\text{old}} \,\|\, \pi_\theta)$
- Sudden performance drops (instability)

Unconstrained Ratio ⇒ Catastrophic Policy Updates

Surrogate objective:

$$L^{\text{PG}}(\theta) = \mathbb{E}\left[r_t(\theta)\, A_t\right], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Issue: unconstrained importance ratios

- No constraint on $r_t(\theta)$
- During SGD:
  - $r_t \gg 1$ for some actions
  - $r_t \ll 1$ for others

Failure modes

- Probability mass collapses onto few actions
- Loss of exploration
- Large KL: $\text{KL}(\pi_{\text{old}} \,\|\, \pi_\theta)$
- Sudden performance drops (instability)

Asymmetry problem

- If $A_t > 0$: $r_t \to \infty$
- If $A_t < 0$: $r_t \to 0$

⇒ **runaway updates**

Unconstrained Ratio ⇒ Catastrophic Policy Updates

Surrogate objective:

$$L^{\mathrm{PG}}(\theta) = \mathbb{E}\left[r_t(\theta)\, A_t\right], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)}$$

Issue: unconstrained importance ratios

- No constraint on $r_t(\theta)$
- During SGD:
  - $r_t \gg 1$ for some actions
  - $r_t \ll 1$ for others

Failure modes

- Probability mass collapses onto few actions
- Loss of exploration
- Large KL: $\mathrm{KL}(\pi_{\mathrm{old}} \,\|\, \pi_\theta)$
- Sudden performance drops (instability)

Asymmetry problem

- If $A_t > 0$: $r_t \to \infty$
- If $A_t < 0$: $r_t \to 0$

⇒ **runaway updates**

Key insight

- Unconstrained ratios make obj. unreliable far from π_{old}
- Need to limit policy change per update

PPO-KL Penalty Objective

Idea: Replace hard trust-region constraint with soft KL penalty and penalize policy deviation directly in the objective

Objective

$$\mathscr{L}^{\text{KL}}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{\text{old}}} \left[r_t(\theta) A_t \; - \; \beta \, D_{\text{KL}} \big(\pi_{\text{old}}(\, \cdot \mid s_t) \, \| \, \pi_\theta(\, \cdot \mid s_t) \big) \right]$$

where $\beta > 0$: KL penalty strength

PPO-KL Penalty Objective

Idea: Replace hard trust-region constraint with soft KL penalty and penalize policy deviation directly in the objective

Objective

$$\mathscr{L}^{\text{KL}}(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{\text{old}}} \left[r_t(\theta) A_t \; - \; \beta \, D_{\text{KL}} \big(\pi_{\text{old}}(\, \cdot \mid s_t) \, \| \, \pi_\theta(\, \cdot \mid s_t) \big) \right]$$

where $\beta > 0$: KL penalty strength

Connection to TRPO

- TRPO:

$$\max_\theta \; \mathbb{E}[r_t(\theta) A_t] \quad \text{s.t.} \quad D_{\text{KL}} \leq \delta$$

PPO-KL:

$$\max_\theta \; \mathbb{E}[r_t(\theta) A_t] \; - \; \beta D_{\text{KL}}$$

- Interpretation: PPO-KL = Lagrangian relaxation of TRPO

Why KL Penalty Is Unstable

- **Core issue**
  - Fixed KL penalty does not work across training
- **Why**
  - Early training → large, noisy updates → need strong regularization
  - Late training → small, stable updates → large β slows learning
  - Same β ⇒ inconsistent policy shifts
- **Practical consequence**
  - β too small → KL explodes / instability
  - β too large → tiny updates / stagnation
- **Heuristic fix**
  - Adapt β online:
    - ↑ if KL too large
    - ↓ if KL too small

PPO-Clip Objective

Idea

- Replace KL penalty with clipped importance ratio
- Prevent objective from exploiting large policy changes

Objective

$$\mathscr{L}^{\text{clip}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,A_t\right)\right]$$

Mechanism

- If $r_t \in [1-\epsilon, 1+\epsilon]$ → standard PG
- Outside → objective is flattened (no incentive to push further)

Key effect

- Implicit trust region
- No KL tuning, no second-order methods

Why Clipping Creates an Approximate Trust Region

- **Trust-Region Effect (Intuition)**
  - The objective **stops increasing** once:

$$r_t(\theta) \notin [1-\epsilon, 1+\epsilon]$$

  - Beyond this region, **gradient signal vanishes** for that sample
  - Therefore, SGD has **no incentive** to push the policy further away from π_{old}

→ This creates a **soft trust region** around π_{old}

Why Clipping Creates an Approximate Trust Region

- **Trust-Region Effect (Intuition)**
  - The objective **stops increasing** once:

$$r_t(\theta) \notin [1-\epsilon, 1+\epsilon]$$

  - Beyond this region, **gradient signal vanishes** for that sample
  - Therefore, SGD has **no incentive** to push the policy further away from π_{old}

→ This creates a **soft trust region** around π_{old}

- **Relation to KL Divergence**
  - For small changes:

$$r_t(\theta) \approx 1 \;\Rightarrow\; D_{\text{KL}}(\pi_{\text{old}} \,\|\, \pi_\theta) \approx 0$$

  - Large KL deviations typically require large ratios r_t
  - By clipping r_t, PPO **indirectly limits KL growth** without explicitly constraining it

PPO Full Loss

Combined loss

$$\mathscr{L}^{\text{PPO}}(\theta, \phi) = \mathscr{L}^{\text{clip}}(\theta) \; - \; c_v \mathscr{L}^{\text{value}}(\phi) \; + \; c_e \mathscr{L}^{\text{entropy}}(\theta)$$

PPO Full Loss

Combined loss

$$\mathscr{L}^{\text{PPO}}(\theta, \phi) = \mathscr{L}^{\text{clip}}(\theta) \; - \; c_v \mathscr{L}^{\text{value}}(\phi) \; + \; c_e \mathscr{L}^{\text{entropy}}(\theta)$$

1. Policy loss (clipped surrogate)

$$\mathscr{L}^{\text{clip}}(\theta) = \mathbb{E}\Big[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \Big]$$

→ controls updates, soft trust region

PPO Full Loss

Combined loss

$$\mathscr{L}^{\text{PPO}}(\theta, \phi) = \mathscr{L}^{\text{clip}}(\theta) \; - \; c_v \mathscr{L}^{\text{value}}(\phi) \; + \; c_e \mathscr{L}^{\text{entropy}}(\theta)$$

1. Policy loss (clipped surrogate)

$$\mathscr{L}^{\text{clip}}(\theta) = \mathbb{E}\Big[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \Big]$$

→ controls updates, soft trust region

2. Value loss (critic)

$$\mathscr{L}^{\text{value}}(\phi) = \mathbb{E}\left[(V_\phi(s_t) - \hat{V}_t)^2 \right]$$

→ fits value function

PPO Full Loss

Combined loss

$$\mathscr{L}^{\text{PPO}}(\theta, \phi) = \mathscr{L}^{\text{clip}}(\theta) \; - \; c_v \mathscr{L}^{\text{value}}(\phi) \; + \; c_e \mathscr{L}^{\text{entropy}}(\theta)$$

1. Policy loss (clipped surrogate)

$$\mathscr{L}^{\text{clip}}(\theta) = \mathbb{E}\Big[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,A_t\right)\Big]$$

→ controls updates, soft trust region

2. Value loss (critic)

$$\mathscr{L}^{\text{value}}(\phi) = \mathbb{E}\left[(V_\phi(s_t) - \hat{V}_t)^2\right]$$

→ fits value function

3. Entropy bonus

$$\mathscr{L}^{\text{entropy}}(\theta) = \mathbb{E}\left[\mathscr{H}(\pi_\theta(\cdot \mid s_t))\right]$$

→ encourages exploration, prevents collapse

- Typical coefficients: $c_v \in [0.5, 1.0]$, $c_e \in [0, 0.01]$, $\epsilon \approx 0.2$

PPO Training Algorithm

```
Initialize policy π_θ and value function V_φ
repeat
    // 1. Collect data
    for t = 1, …, T do
        sample a_t ~ π_old(· | s_t)
        observe r_t, s_{t+1}
        store (s_t, a_t, r_t, log π_old(a_t | s_t))
    end for
    // 2. Compute targets
    compute advantages Â_t   (e.g., GAE-λ)
    compute value targets V̂_t = Â_t + V_φ(s_t)
    // 3. Optimize (multiple epochs over same data)
    for epoch = 1, …, K do
        shuffle data and split into minibatches
        for each minibatch do
            r_t(θ) ← π_θ(a_t | s_t) / π_old(a_t | s_t)
            L^clip ← E_t[min(r_t(θ) Â_t,  clip(r_t(θ), 1-ε, 1+ε) Â_t)]
            L^vf  ← E_t[(V_φ(s_t) - V̂_t)²]
            L^PPO ← -L^clip + c_1 L^vf - c_2 H[π_θ]
            update θ, φ via SGD on L^PPO
        end for
    end for
    // 4. Update old policy
    π_old ← π_θ
until convergence
```

📘 PPO performs multiple epochs on the same data (allows sample efficiency)

Practical Tricks in PPO

1. Advantage normalization

$$\hat{A}_t \leftarrow \frac{\hat{A}_t - \mu(\hat{A})}{\sigma(\hat{A}) + \epsilon}$$

- stabilizes updates
- prevents large-advantage domination

2. Reward scaling / normalization

- normalize or rescale rewards across tasks
- stabilizes critic + policy gradients

3. Value function clipping

$$\mathscr{L}^{\text{value}}_{\text{clip}} = \mathbb{E}\left[\max\left((V_\phi - \hat{V})^2, (\text{clip}(V_\phi, V_{\text{old}} \pm \epsilon_v) - \hat{V})^2\right)\right]$$

- avoids large critic updates
- keeps value aligned with policy

4. Entropy scheduling

- high entropy early → exploration
- anneal $c_e \to 0$ → exploitation

5. KL monitoring (even with clipping)

- track empirical KL
- use for debugging / early stopping

PPO Failure Modes

- Clip range too large
  → unstable updates, KL spikes, collapse

- Clip range too small
  → overly conservative, slow learning

- Too many epochs per batch
  → policy drift despite clipping

- No KL monitoring / late stopping
  → silent policy divergence

- Entropy collapses too early
  → poor exploration, suboptimal policy

- Weak critic
  → noisy / wrong advantages

Why PPO Powers Modern Sequence RL

Stable updates

- Soft trust region → prevents destructive policy shifts

Sample efficiency

- Multiple epochs per batch → reuse expensive rollouts

Long-horizon credit assignment

- Advantages + GAE → effective sequence-level learning

First-order & scalable

- Pure SGD → no second-order ops, GPU-friendly

Flexible regularization

- KL, entropy, reward shaping → control behavior & alignment

Summary

What we learned

- **Policy gradients** optimize policies via expected returns
- **Actor–Critic** reduces variance with learned value functions
- **GAE** enables a principled bias–variance tradeoff
- **TRPO:** trust regions + theory, but expensive
- **PPO:** practical trust regions via clipping + first-order methods
- **Modern RL (LLMs)** relies on PPO-style training for stability

Big picture

REINFORCE → Actor–Critic → TRPO → PPO

- Shift: theory → scalability
- PPO = sweet spot for large-scale RL

📘 **Stable policy optimization is about controlling how much the policy changes — not just following the gradient.**

Recommended Reading

- **Reinforcement Learning: An Introduction**
  Ch. 13–14 — Policy Gradient & Actor–Critic

- **Schulman et al. (2015)**
  Trust Region Policy Optimization (TRPO)

- **Schulman et al. (2017)**
  Proximal Policy Optimization Algorithms (PPO)

- **Schulman et al. (2015)**
  High-Dimensional Continuous Control Using GAE